

Challenge Write-Up Template

Challenge Information

Student Name	Pascua, Diana KC S.
Section / Class	BSIT-3G
Challenge Name	Ping-cmd
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/14/26
Flag Format Given	picoCTF{....}

1. Challenge Description

This challenge connected me to a small service over netcat that claimed to let me ping an IP address, but insisted it had "tight security" because it only allowed [8.8.8.8](#). The hints made the actual goal clear even before I touched anything: the program was quietly building a shell command out of whatever I typed, and my job was to sneak a second command in alongside the one it expected.

2. Initial Analysis / Reconnaissance

The two hints on the challenge page did most of the thinking for me before I even connected: "The program uses a shell command behind the scenes," and "Sometimes, you can run more than one command at a time." Together those pointed straight at OS command injection — a program taking user input and passing it, unsanitized, into a call like `ping -c 2 <input>`. Since shells let you chain commands together with operators like `;`, `&&`, or `|`, whatever I typed after the expected IP address wouldn't just be ignored — it would get executed as its own command. That told me this was a classic injection challenge, not anything cryptographic or file-based.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
nc mysterious-sea.picoctf.net 58728	Connect to the challenge service.	Prompted: "Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8')"
8.8.8.8 ; cat flag.txt	Test whether the input gets passed straight into a shell command.	Ping ran normally against 8.8.8.8 , and right after it, flag.txt's contents printed too.

4. Solution Walkthrough

Step 1

Connect and see what the service expects

I ran `nc mysterious-sea.picoctf.net 58728` and got the prompt: "Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8')." The wording of that "security" claim was a giveaway on its own — allowlisting a single fixed IP is only meaningful if the rest of my input isn't being filtered at all, which is exactly the kind of false confidence command-injection challenges like to bait you with.

Step 2

Confirm the allowed value still works

Following the hints, my working theory was that the server was doing something like running `ping -c 2 8.8.8.8` by string-concatenating whatever I sent onto that command. If that was true, anything I appended after `8.8.8.8`, separated by a shell metacharacter, should run as a second, independent command.

Step 3

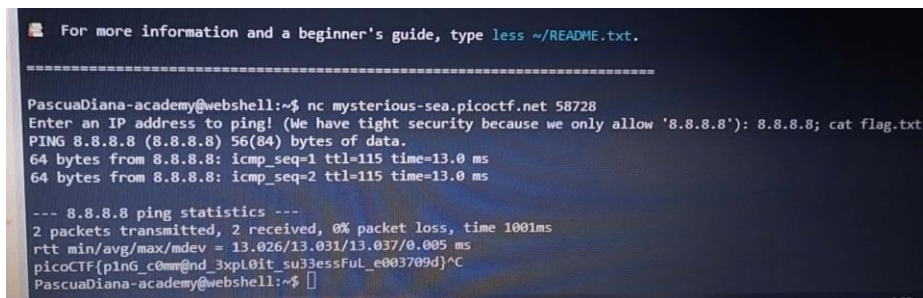
Inject a second command

I sent `8.8.8.8; cat flag.txt`. The semicolon in a shell means "run the next command regardless of whether the previous one succeeded," so this line asked the server to first ping `8.8.8.8` exactly like it expected, and then, separately, run `cat flag.txt`. The output showed exactly that: two normal ping replies from `8.8.8.8`, followed immediately by the contents of `flag.txt` printed straight into the terminal.

Step 4

Grab the flag

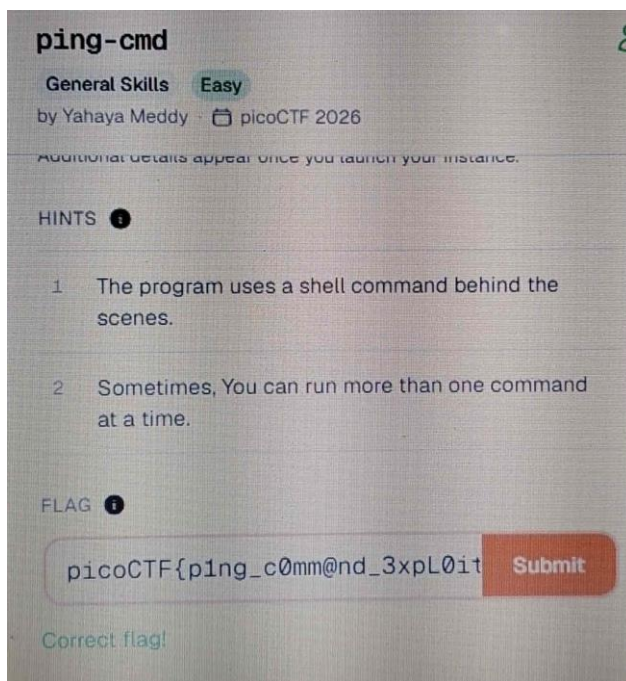
The flag appeared directly in the netcat output right after the ping statistics: `picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_e003709d}`, which I copied and submitted for a confirmed "Correct flag!"



```
For more information and a beginner's guide, type less ~/README.txt.

=====
PascuaDiana-academy@webshell:~$ nc mysterious-sea.picoctf.net 58728
Enter an IP address to ping! (We have tight security because we only allow '8.8.8.8'): 8.8.8.8; cat flag.txt
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=13.0 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=13.0 ms

--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 13.026/13.031/13.037/0.005 ms
picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_e003709d}^C
PascuaDiana-academy@webshell:~$
```



5. Flag

picoCTF{p1nG_c0mm@nd_3xpL0it_su33essFuL_e003709d}

6. Key Concept / What I Learned

This challenge is a hands-on example of OS command injection: a vulnerability class where an application builds a shell command by directly inserting untrusted user input, instead of passing that input as a properly separated argument. Allowlisting a single expected value (like [8.8.8.8](#)) doesn't help if the rest of the string still reaches a shell unfiltered, since shell metacharacters like ; can be used to chain on an entirely new command. The real fix is to never build shell commands from raw user input — use APIs that take arguments as a list (avoiding a shell entirely), or strictly validate and escape input if a shell call is unavoidable.

7. References
